

Universidad Católica de Temuco

Ingeniería Civil en Informática

Reimplementación compatible de OpenStack Glance en un lenguaje compilado

Etapla 2: análisis, diseño y línea base

Estudiante: Juan Cardenas Uribe

Profesor guía: Alejandro Mellado Gatica

Curso: Trabajo de Título (INFO-1197)

Fecha: 2 de octubre de 2026

Índice

1	Introducción	2
2	Definición del problema	2
2.1	Contexto	2
2.2	Problema identificado	2
2.3	Relevancia e impacto	3
2.4	Partes interesadas y percepción del problema	3
2.5	Alcance y límites	3
3	Objetivos	4
3.1	Objetivo general	4
3.2	Objetivos específicos	4
3.3	Verificación SMART	4
3.4	Matriz de trazabilidad problema–objetivos	5
4	Análisis de requerimientos	5
4.1	Base técnica y arquitectura del servicio	5
4.2	Selección del servicio y del lenguaje	6
4.3	Entorno de despliegue: DevStack versus instalación manual	6
4.4	Contrato Image API v2	7
4.5	Modelo de datos y ciclo de vida	7
4.6	Flujo de datos e integración con Nova	8
4.7	Requerimientos funcionales	9
4.8	Requerimientos no funcionales	9
4.9	Diseño objetivo y criterios de aceptación	10
4.10	Riesgos y mitigaciones	10
5	Planificación y backlog	10
5.1	Épicas	10
5.2	Backlog prioritario y estado del Sprint 2	11
5.3	Plan por sprints e hitos	11
6	Conclusiones de la Etapa 2	12
	Referencias	12

1 Introducción

OpenStack Glance permite registrar, consultar y distribuir imágenes utilizadas para crear instancias. El servicio combina un catálogo de metadatos con almacenamiento de archivos binarios y acceso autenticado. Esa combinación permite estudiar una reimplementación que debe preservar tanto la API como la integridad de los datos [1, 2].

Este informe corresponde al trabajo de Juan Cardenas Uribe y concentra el análisis en Glance. Conserva la organización del documento de Etapa 1: definición del problema, objetivos, requerimientos, planificación y conclusiones. En la Etapa 2 se agregan arquitectura, contrato Image API, persistencia, flujo de datos y evidencia de la línea base.

Se propone Go para una implementación acotada de Image API v2. La evaluación posterior deberá comparar compatibilidad, latencia, throughput y recursos. No se supone una mejora automática por el cambio de lenguaje. Esta etapa verifica el servicio original y define el diseño; aún no existe una implementación Go aceptada ni benchmarks comparativos.

El laboratorio es común con el trabajo de Placement. Ese servicio permanece como dependencia original al evaluar Glance. Los resultados de instalación y smoke tests se presentan como evidencia compartida, sin atribuir su ejecución exclusivamente al autor de este informe.

2 Definición del problema

2.1 Contexto

Nova necesita información y datos de imagen para crear una instancia. Glance expone Image API v2, aplica autenticación y políticas, conserva metadatos en una base de datos y delega el almacenamiento binario en un backend. Metadatos y bytes deben mantenerse coherentes a través del ciclo de vida de la imagen [2, 1].

La línea base proviene de DevStack de un nodo, con un único backend filesystem. Permite probar registro, actualización, transferencia, eliminación e integración con Nova, sin introducir la complejidad de almacenamiento distribuido.

2.2 Problema identificado

Una reimplementación que almacena registros de imagen pero altera estados, formatos, políticas o bytes puede responder a la API y aun así impedir el boot de una instancia. La transferencia de imágenes también puede consumir memoria de forma distinta según el diseño. El problema es conservar un ciclo mínimo compatible y evaluar su costo bajo condiciones controladas.

La pregunta de investigación es: *¿es posible reimplementar el alcance seleccionado de Glance en Go, manteniendo la gestión de imágenes y su consumo por Nova, y qué diferencias de rendimiento y recursos presenta frente al servicio original?*

2.3 Relevancia e impacto

El resultado esperado incluye un contrato reproducible de Image API v2 y un prototipo que integre metadatos, persistencia y transferencia binaria. La investigación podrá distinguir los costos de procesamiento de metadatos de los costos de transferencia y almacenamiento. Las conclusiones deberán reportar mejoras, regresiones y límites del backend seleccionado.

2.4 Partes interesadas y percepción del problema

Cuadro 1: Partes interesadas del proyecto Glance.

Parte interesada	Necesidad o interés
Estudiante responsable	Juan Cardenas Uribe: delimitar, diseñar e implementar Glance y sustentar su evaluación.
Profesor guía	Validar la coherencia entre alcance, resultados y evidencia del trabajo de título.
Usuarios y clientes API	Gestionar imágenes con respuestas, estados y reglas compatibles.
Nova	Consumir metadatos y datos de imagen para el escenario de boot comprometido.
Equipo del laboratorio	Repetir instalación y pruebas; aislar la sustitución de Glance.

2.5 Alcance y límites

El alcance incluye crear, listar, consultar, actualizar y eliminar metadatos; cargar y descargar datos binarios; conservar autenticación requerida por el laboratorio; y permitir que Nova cree una instancia desde una imagen servida por el reemplazo.

Se utiliza un único backend de archivos local. Multi-store, importación interoperable completa, replicación, federación, caché distribuida y metadefs completos quedan fuera del alcance inicial, salvo que una prueba de integración demuestre una dependencia necesaria. Se deben definir límites de formato, visibilidad y permisos a partir de los casos comprometidos.

El original anunció Image API v2.18. Esa versión describe la API de Glance y no se trata como la negociación de microversiones de Placement. Keystone, Nova, Placement, Neutron y la virtualización permanecen como componentes originales. La evaluación no representa producción ni escalado multinodo.

3 Objetivos

3.1 Objetivo general

Diseñar, implementar y evaluar una reimplementación compatible del subconjunto seleccionado de OpenStack Glance en Go, que permita gestionar y transferir imágenes e integrarse con Nova, comparando su funcionamiento, rendimiento y recursos con el servicio original.

3.2 Objetivos específicos

Los identificadores OE1–OE5 mantienen la trazabilidad de la planificación común; la aplicación de cada objetivo se concentra aquí en Glance. El hito de 50% de implementación corresponde al proyecto conjunto; el Excel ubica las historias B20 y B25 de Glance en Sprint 4.

- OE1.** Analizar arquitectura, Image API, autenticación, persistencia y flujo de datos antes del 2 de octubre de 2026.
- OE2.** Disponer del laboratorio reproducible y verificar la línea base del ciclo de imagen y el boot original antes del 2 de octubre de 2026.
- OE3.** Contribuir al hito común de al menos 50% de implementación del proyecto al 30 de octubre de 2026, preparando el esqueleto y fixtures de Glance para las historias de Sprint 4.
- OE4.** Integrar y estabilizar gestión de metadatos, transferencia binaria y consumo por Nova antes del 20 de noviembre de 2026.
- OE5.** Comparar compatibilidad, latencia, throughput, CPU, memoria y errores, y presentar conclusiones antes del 4 de diciembre de 2026.

3.3 Verificación SMART

Cuadro 2: Objetivos Glance, indicadores y estado al cierre de Sprint 2.

ID	Indicador y evidencia	Plazo	Estado
OE1	Arquitectura, endpoints, datos, estados, backend y flujo de integración documentados.	02-10-2026	Cumplido en S2.
OE2	Ciclo original probado, descarga idéntica, rechazo sin token y boot ACTIVE.	02-10-2026	Cumplido en S2.
OE3	Hito común: 50% del proyecto; Glance prepara esqueleto y fixtures para su implementación.	30-10-2026	Planificado.
OE4	Nova crea una instancia usando la imagen del reemplazo; casos críticos estabilizados.	20-11-2026	Planificado.
OE5	Dataset de metadata/transferencia, repeticiones y análisis comparativo.	04-12-2026	Planificado.

El cumplimiento de OE1 se limita a la especificación de Sprint 2. No implica haber verificado cada política, error o estado de la API completa; esos casos se ampliarán durante el desarrollo.

3.4 Matriz de trazabilidad problema–objetivos

Cuadro 3: Trazabilidad del problema Glance.

Aspecto del problema	Objetivo	Resultado verificable
Contrato, estados y almacenamiento	OE1	Inventario Image API, modelo y separación metadata/bytes.
Pruebas difíciles de repetir	OE2	Laboratorio automatizado y ciclo de imagen original documentado.
Integridad y consumo por Nova	OE3/OE4	Transferencia idéntica, persistencia y boot con Glance Go.
Eficiencia sin evidencia	OE5	Mediciones separadas de operaciones metadata y transferencia.

4 Análisis de requerimientos

4.1 Base técnica y arquitectura del servicio

El servicio desplegado separa API HTTP, autenticación/políticas, controladores y dominio, persistencia de metadatos y `glance_store`. Los archivos de configuración y las tablas reales se contrastaron con los modelos del código fijado. El backend confirmado usa `/opt/stack/data/glance/images/` [4].

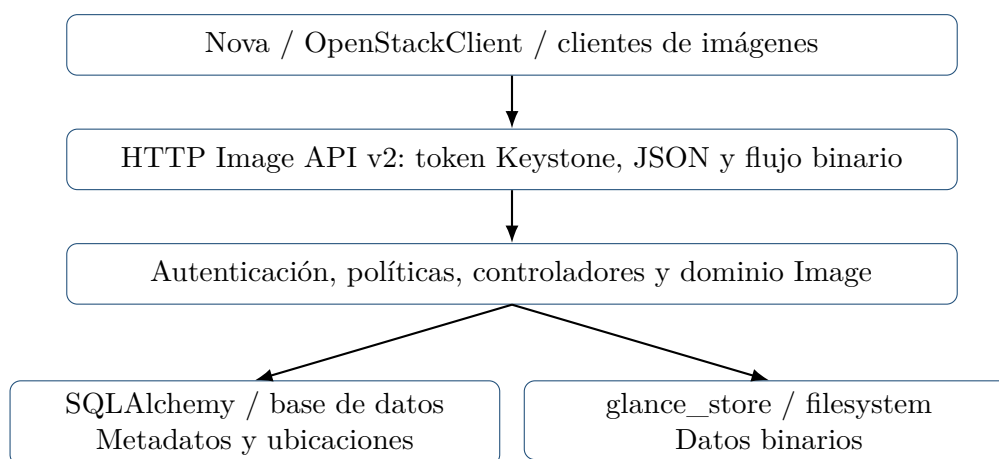


Figura 1: Separación de metadatos y datos de imagen en el servicio original.

4.2 Selección del servicio y del lenguaje

Glance aporta un caso de investigación que combina llamadas de metadatos, persistencia y transferencia de archivos. Un alcance limitado a Image API v2 y filesystem permite evaluar esa combinación sin implementar todos los backends de OpenStack.

Go se selecciona por su soporte HTTP y de concurrencia, compilación a binario y facilidad para organizar adaptadores de almacenamiento. El beneficio de streaming sobre memoria o throughput debe verificarse; no se atribuye automáticamente al lenguaje.

Cuadro 4: Alternativas de lenguaje para Glance.

Criterio	Go	Rust	C++
API y transferencia	HTTP e interfaces de streaming disponibles.	Buen soporte; mayor curva inicial.	Requiere integrar bibliotecas.
Memoria y concurrencia	Gestión automática y goroutines.	Propiedad y seguridad estática.	Control manual más complejo.
Riesgo en el plazo	Moderado.	Mayor costo inicial estimado.	Mayor costo de integración estimado.
Decisión	Seleccionado.	Alternativa.	No priorizado.

La comparación expresa la valoración de diseño del proyecto, no una medición experimental de rendimiento entre lenguajes.

4.3 Entorno de despliegue: DevStack versus instalación manual

Se utiliza DevStack para automatizar instalación y configuración en un laboratorio de desarrollo [3]. Una instalación manual ofrecería más control por componente, con mayor tiempo y documentación de configuración. Terraform y Ansible hacen repetible la alternativa elegida.

La VM compartida `tt-openstack-lab` tiene Ubuntu 24.04.5, 4 vCPU, 8 GiB de RAM y disco de 55 GiB. Se verificaron Keystone, Nova, Placement, Glance, Neutron, MySQL y RabbitMQ. El smoke test utiliza QEMU emulado; una segunda ejecución de Ansible terminó sin reinstalar DevStack.

Cuadro 5: Versiones registradas en la línea base Glance.

Componente	Versión o commit
DevStack	<code>stable/2026.2;</code> <code>0fb9685c1bf22cd0b4d576d7a74f64195ad934fe.</code>
Glance	<code>a3e5f490b7a28b1161714c94acffe5308d24b567.</code>
Nova	<code>247ea5d774305b688997f8df81c011b249eb793c.</code>
Python / MySQL	3.12.3 / 8.0.46.
Image API / OpenStackClient	v2.18 anunciada / 10.3.0.

La configuración y un parche Geneve están versionados. Las dependencias secundarias no se fijaron para una reconstrucción bit a bit. La VM quedó apagada por solicitud del usuario después de registrar la evidencia, conservando sus discos; no se ejecutaron pruebas nuevas para elaborar este documento.

4.4 Contrato Image API v2

La API utiliza tokens Keystone. Metadata usa JSON; la carga binaria emplea `application/octet-stream`; PATCH utiliza `application/openstack-images-v2.1-json-patch`. Los resultados de la línea base original se detallan a continuación.

Cuadro 6: Operaciones Glance y resultados de la línea base.

Método y ruta	HTTP	Resultado observado
POST /v2/images	201	Imagen registrada en estado queued.
GET /v2/images	200	Listado de imágenes autenticado.
GET /v2/images/{id}	200	Consulta de metadatos y estado.
PATCH /v2/images/{id}	200	Actualización de metadatos mediante JSON Patch.
PUT /v2/images/{id}/file	204	Carga de bytes; consulta posterior en active.
GET /v2/images/{id}/file	200	Descarga idéntica byte a byte y por SHA256.
DELETE /v2/images/{id}	204	Eliminación del recurso temporal.
GET /v2/images/{id} ausente	404	Recurso inexistente tras eliminación.
GET /v2/images sin token	401	Rechazo de la petición no autenticada.

Las políticas de visibilidad, propietario y roles necesitan fixtures específicos de 403 y acceso permitido. También faltan casos de PATCH inválido, transferencia interrumpida, límites de tamaño y conflicto. El éxito del camino principal no acepta automáticamente estos comportamientos del reemplazo.

4.5 Modelo de datos y ciclo de vida

Los metadatos de `images` incluyen UUID, nombre, propietario, estado, visibilidad, formatos, tamaño, checksum, hashes, mínimos de disco/RAM, protección y timestamps. Los bytes se almacenan fuera de la base de datos. La auditoría confirmó las tablas; el detalle de claves proviene de los modelos SQLAlchemy fijados.

Cuadro 7: Entidades y almacenamiento del dominio Glance.

Entidad	Datos centrales	Regla de diseño
Image	UUID PK, propietario, estado, formatos, tamaño y hashes.	Identidad y ciclo de vida consistentes.
ImageProperty	ID, image_id, nombre y valor; unicidad imagen/nombre.	Actualizar propiedades con validación.
ImageLocation	ID, image_id, ubicación, metadata y estado.	Relacionar ubicación con el backend autorizado.
ImageTag / ImageMember	Etiquetas y membresía de acceso.	Delimitar operaciones según los casos necesarios.
Store filesystem	Archivo binario separado del catálogo.	Integridad, permisos y limpieza de datos.

Se observó queued tras crear la imagen y active después de cargar los datos. Saving y killed forman parte del análisis del ciclo de carga, pero no se observaron en pruebas de fallo de esta etapa; deben comprobarse antes de aceptar recuperación de errores.

Se propone un esquema propio acotado para metadatos y un adaptador filesystem. El diseño escribe a un archivo temporal, calcula tamaño/hash y publica mediante rename atómico antes de confirmar active. Como base de datos y filesystem no comparten una transacción, se necesita reconciliación y limpieza ante fallos. Esa recuperación es una tarea de implementación y prueba, no una propiedad ya demostrada.

4.6 Flujo de datos e integración con Nova

El flujo mínimo verificado fue registro de imagen, carga de bytes, consulta, modificación de metadata, descarga, comparación y eliminación. El baseline preservó requests, respuestas, status y headers saneados. La igualdad de bytes y SHA256 sustenta la integridad del caso original probado.

El smoke test creó una instancia desde CirrOS y registró la transición BUILD a ACTIVE. La instancia fue eliminada después de la verificación. Este resultado comprueba la integración general del OpenStack original, pero no captura la conversación HTTP interna Nova–Glance ni permite afirmar qué descarga o uso de caché ejecutó Nova.

Para aceptar el reemplazo, Nova deberá consultar y consumir una imagen gestionada por Go. Se capturará el flujo real y se controlará el estado de caché de la prueba. Placement permanecerá original en ese escenario para aislar las diferencias del módulo Glance.

4.7 Requerimientos funcionales

Cuadro 8: Requerimientos aplicables al módulo Glance.

ID	Requerimiento	Verificación prevista
RF01	OpenStack de un nodo reproducible.	Instalación y smoke test; base validada en S2.
RF07	Persistencia consistente de metadata y ubicaciones.	Reinicio, fallos y reconciliación con store.
RF08	Autenticación requerida por OpenStack.	Casos autorizados, 401 y 403.
RF10	Registrar, listar, consultar, modificar y eliminar imágenes.	Comparación diferencial de Image API v2.
RF10	Cargar/descargar bytes y mantener estados e integridad.	Comparación byte a byte/hash y pruebas de fallos.
RF10	Permitir consumo de imagen por Nova.	Boot usando el reemplazo Glance.
RF11	Observabilidad del módulo.	Logs saneados y diagnóstico de transferencias/errores.
RF12	Automatización funcional y de carga.	Scripts versionados y dataset repetible.

RF10 es el requisito específico de Glance en el Excel. Las filas separan sus criterios para hacerlos verificables sin crear nuevos IDs. RF07/RF08 y RF11/RF12 aportan capacidades transversales. La implementación Go sigue pendiente.

4.8 Requerimientos no funcionales

Cuadro 9: Atributos de calidad para el módulo Glance.

ID	Atributo	Criterio
RNF01	Compatibilidad	Comparar status, metadata, estados, errores, headers y efectos persistidos.
RNF02	Rendimiento	Latencia de metadata y throughput de transferencia bajo cargas separadas.
RNF03	Recursos	CPU y memoria en reposo, carga y transferencias de distintos tamaños.
RNF04	Fiabilidad	Repeticiones, tasa de error y recuperación de cargas interrumpidas.
RNF05	Reproducibilidad	Versiones, backend, configuración, comandos y dataset registrados.
RNF06	Mantenibilidad	API, dominio, repositorio y store desacoplados, con pruebas.
RNF07	Seguridad	Conservar auth/políticas; controlar rutas/permisos y evitar secretos en fixtures.

4.9 Diseño objetivo y criterios de aceptación

El ejecutable propuesto es `cmd/glance`. Se separan paquetes API, servicio, modelo y repositorio, con `glance/store` para Put/Get/Delete de bytes. Configuración, autenticación Keystone y logging son componentes compartidos. El store deberá transferir datos mediante streaming y evitar cargar el archivo completo en memoria; su comportamiento se medirá.

La aceptación requiere que original y Go superen el ciclo mínimo, que los datos descargados sean idénticos a los cargados, que estados y permisos sean compatibles y que Nova cree una instancia desde la imagen del reemplazo. Se compararán respuestas y estado con semillas equivalentes. UUID, timestamps y request IDs podrán normalizarse sin ocultar diferencias de contenido o visibilidad.

Las pruebas se organizan en unidad, contrato API, comparación diferencial, persistencia y store, integración y boot completo. Los benchmarks separarán metadata de transferencia y controlarán tamaño de imagen, concurrencia, disco, caché, warm-up y repeticiones. QEMU y el único filesystem limitan la generalización de resultados. No se presentan métricas comparativas de Go en esta etapa.

4.10 Riesgos y mitigaciones

Cuadro 10: Riesgos del desarrollo Glance.

Riesgo	Nivel	Mitigación
Alcance de Image API demasiado amplio	Alto	Conservar ciclo mínimo y un solo backend; ampliar con evidencia de Nova.
Metadata active sin bytes completos	Alto	Archivo temporal, publicación atómica y recuperación/reconciliación probada.
Autorización o visibilidad incorrectas	Alto	Fixtures de propietario, roles, 401/403 y acceso permitido.
Benchmark condicionado por disco/caché	Medio	Controlar estado, tamaño de archivo y carga; separar métricas.
Flujo interno Nova no capturado	Medio	Captura saneada y boot controlado antes de aceptar sustitución.
Dependencias o feedback pendientes	Medio	Registrar versiones y añadir observaciones formales del profesor.

5 Planificación y backlog

5.1 Épicas

Aplican las épicas del Excel vigente E1 infraestructura y entorno; E2 análisis y compatibilidad; E4 integración OpenStack; E5 pruebas y benchmarking; E6 reimplementación Glance. E3 reimplementación Placement corresponde al informe independiente de ese módulo. La documentación y la infraestructura común conservan su trazabilidad en ambos informes.

5.2 Backlog prioritario y estado del Sprint 2

Los identificadores corresponden al Excel vigente [5]. El backlog preliminar del PDF de Etapa 1 fue refinado; no se utiliza su antigua numeración para asociar pruebas actuales.

Cuadro 11: Elementos de Sprint 2 relevantes para Glance.

ID	Elemento	SP	Estado
B01	Preparar VM de laboratorio (compartido).	3	Hecho: VM auditada.
B02	Instalar DevStack de un nodo (compartido).	5	Hecho: endpoints verificados.
B03	Smoke tests del original (compartido).	5	Hecho: boot ACTIVE y limpieza.
B19	Analizar arquitectura, Image API, auth, persistencia y flujo de Glance.	5	Hecho: especificación y baseline.

La tarea específica de análisis Glance suma 5 SP; las tareas comunes, 13 SP. El Sprint 2 conjunto alcanzó 7/7 elementos y 33/33 SP al incluir B04–B06 de Placement. Esto describe aceptación de la planificación compartida, no productividad individual ni porcentaje implementado de Glance.

Las capturas están en `evidence/sprint2/glance/`, incluidas auditoría de backend, resultados de API y `image-download.sha256.txt`. El boot está en `evidence/sprint2/tests/boot-states.txt`; el modelo y la arquitectura se documentan en `docs/sprint2/`. El registro `infra/devstack/versions.lock` fija el código inspeccionado. La verificación `make sprint2-verify` aprobó las evidencias requeridas.

5.3 Plan por sprints e hitos

Cuadro 12: Hitos del módulo Glance.

Periodo	Hito	Resultado esperado
Hasta 04-09	Etapa 1	Problema, objetivos y alcance mínimo de imágenes.
05-09 a 02-10	Etapa 2 / S2	Arquitectura, API, estados, backend y baseline original.
03-10 a 30-10	Etapa 3 / S3	Hito común de 50% del proyecto; preparar esqueleto, contratos y fixtures de Glance. B20/B25 permanecen en S4.
31-10 a 20-11	Cierre / S4	Implementar catálogo mínimo (B20) y carga/descarga (B25); integrar auth, recuperación y boot con Nova.
21-11 a 04-12	Evaluación / S5	Comparación metadata/transferencia, dataset, conclusiones e informe final.

6 Conclusiones de la Etapa 2

Glance quedó delimitado a un ciclo mínimo de Image API v2 y un store filesystem. La evidencia original confirma registro, consulta, modificación, carga, descarga íntegra y eliminación, junto con el rechazo sin token. El smoke test de OpenStack alcanzó ACTIVE desde CirrOS.

OE1 y OE2 se cumplen para el alcance de Sprint 2. La integración específica del reemplazo Go, las políticas adicionales, los estados de fallo y los benchmarks permanecen como trabajo posterior. No se infiere equivalencia completa a partir del ciclo principal ni se atribuye a Go una mejora aún no medida.

El siguiente paso es implementar metadatos y persistencia, desarrollar el store con streaming y recuperación, y comparar cada operación con la línea base. La captura del flujo Nova–Glance y la retroalimentación formal del profesor completarán la evidencia a medida que avance el desarrollo.

Referencias

- [1] OpenStack. *Image service overview – Glance*. Referencia utilizada en el informe de Etapa 1. <https://docs.openstack.org/glance/latest/install/get-started.html>.
- [2] OpenStack. *Image Service API v2*. <https://docs.openstack.org/api-ref/image/v2/>.
- [3] OpenStack. *DevStack Documentation*. <https://docs.openstack.org/devstack/latest/>.
- [4] Proyecto *modules-devstack-optimized*. Documentación y evidencias Sprint 2, capturadas el 2 de octubre de 2026. Directorios `docs/sprint2/`, `evidence/sprint2/` e `infra/`; registro de evidencia y matriz final de trazabilidad.
- [5] *Planificacion_Trabajo_Titulo_OpenStack_Placement_Glance.xlsx* y copia de seguimiento Sprint 2. Hojas Objetivos_SMART, Requerimientos, Backlog, Trazabilidad y Sprints. La copia original se mantiene inalterada.